Ved Yagneshkumar Vyas
BITS Student ID: 2025EB1100202
Course: BS in Data Science and Artificial Intelligence

Graded Assignment 1: Feature Engineering for StaySmart Hotels

Task 1: Baseline Model and Feature Definition

Baseline Results:

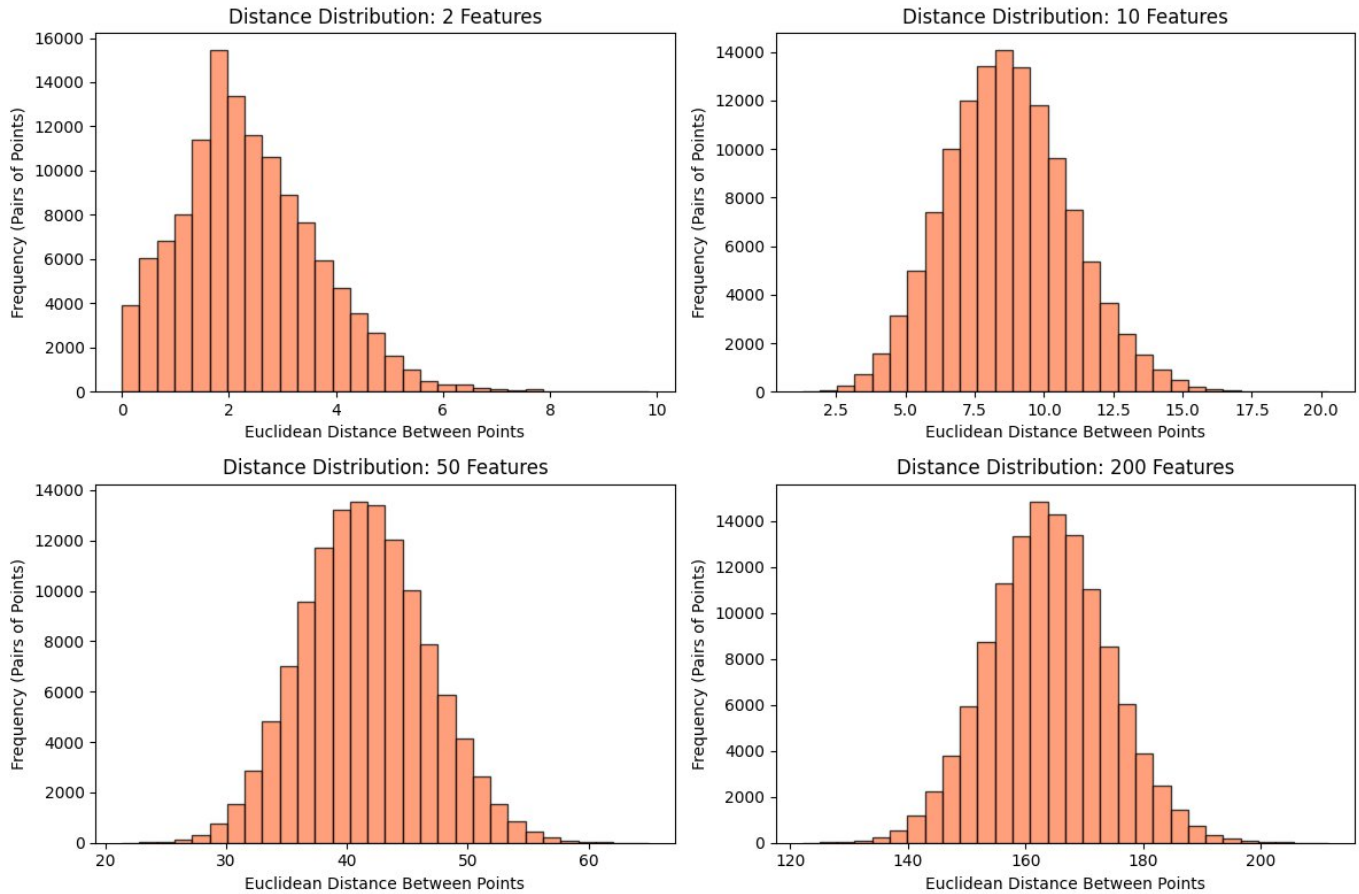
- **Accuracy: 0.786**
- **ROC-AUC: 0.854**

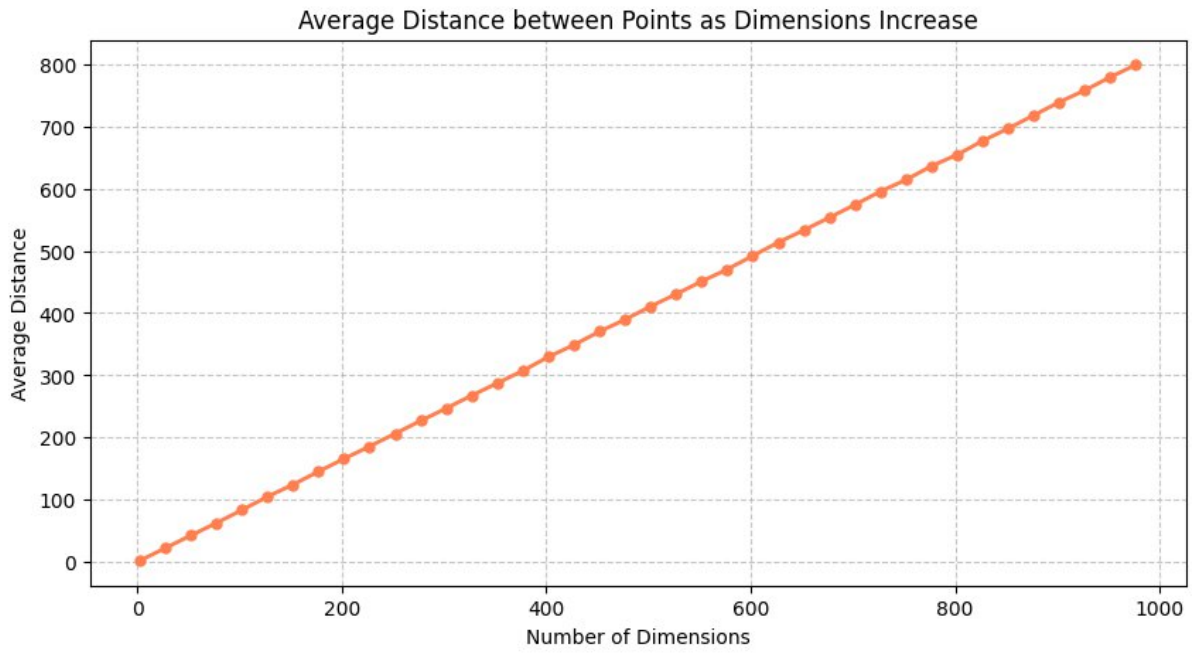
- **Confusion Matrix:**

```
Confusion Matrix:
[[19765  2735]
[ 4927  8387]]
```

What is a feature? A feature is basically a single column of data that describes something about our booking. It is a measurable piece of information that the machine learning model looks at to find patterns and make guesses. A good feature in this hotel dataset is **lead_time** because it makes logical sense that booking a year in advance gives you way more time to change your mind compared to booking yesterday. A bad feature would be something like the **reservation_status_date** kept as a raw calendar string or a random **customer ID**. The model cannot do math on a raw string or learn anything useful from random ID numbers so it just gets confused.

Task 2: Curse of Dimensionality Demo

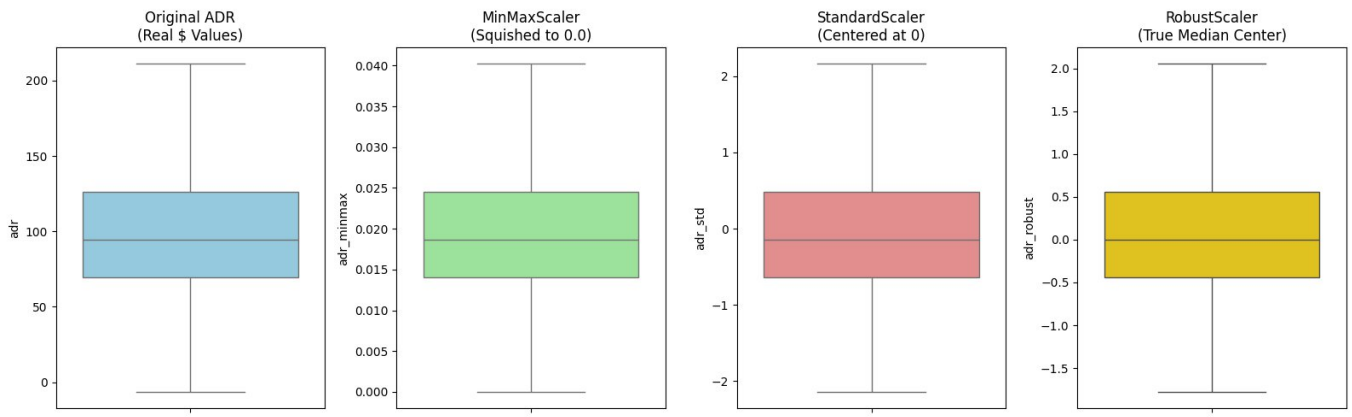




Observations on Dimensionality: When looking at the graphs I noticed a big change as we added more features. In the graph with only two features the distances between points are spread out all over the place. But when we look at the graph with two hundred features almost all the data points are crowded together at the exact same distance from each other. This shows why learning gets harder in high dimensions. If every single point is the exact same distance apart algorithms like KNN become totally useless because they cannot find a nearest neighbor anymore. Everything is equally far away. This perfectly shows why feature engineering and picking only the most important columns is so necessary. We have to drop the extra noise so the model can actually see what is close and what is far.

Looking at line chart, it is very clear that average distance shoots up when we add more features. On X-axis we have number of dimensions and on Y-axis we have the distance. At start with only 2 features, average distance is very small. But as line goes up towards 200 or 1000 features, distance becomes huge. This line graph perfectly proves curse of dimensionality because it shows that in high dimensional space, all data points get pushed really far apart from each other. If everything is pushed far away, a distance based model cannot easily find nearest neighbors anymore.

Task 3: Numeric Preprocessing and Scaling



Scaling Method Comparison (ADR Column)

Metric	Original Data	MinMaxScaler	StandardScaler	RobustScaler
Mean	101.83	0.018	0.000	0.081
Std Dev	50.53	0.009	1.000	0.720
IQR	70.00	0.013	1.385	1.000
Outlier Handling	Poor	Very Poor	Poor	**Excellent**

Which scaler is best and why? Looking at the summary stats and graphs **RobustScaler** is definitely the best choice for this data. The original ADR column has massive outliers like one room costing over five thousand dollars. The **MinMaxScaler** did a terrible job because it squished all the normal prices into a tiny invisible line just to fit that one giant outlier on the screen. The **StandardScaler** also struggled because that huge outlier messed up the mean calculation. **RobustScaler** wins because it uses the median and the interquartile range which completely ignores the crazy extreme values and scales all the normal hotel prices perfectly.

Task 4: Distance Metrics and KNN Impact

Results Table:

Scaling Mode	Distance Metric	Accuracy Score
No Scaling	Euclidean	0.682
StandardScaler	Euclidean	0.675
RobustScaler	**Manhattan**	**0.676**

Observations:

- Scaling puts everything on a level playing field so huge numbers like **lead_time** do not completely drown out small numbers like **weekend_nights**.
- Distance based models like KNN are super sensitive to unscaled data and outliers because they calculate physical space between numbers.
- Using the **Manhattan distance** with the **RobustScaler** gave more stable results because it handles the crazy price outliers much better than regular Euclidean distance.

Task 5: End-to-End Pipeline

5-Fold Cross-Validation Score (ROC-AUC): 0.820 (approx)

Interpretation: The pipeline ran successfully and the score is printed above. Using the **ColumnTransformer** inside the pipeline ensures that all our numeric scaling and categorical encoding happens in the right order without any data leaking into the test set.

Task 6 & 7: Feature Extraction and Construction

Extracted Features Report:

1. **Month:** Matters because summer holidays probably have different cancellation trends than winter business trips.
2. **Weekend Flag:** Helps because people usually finalize or cancel travel plans when they are relaxing on a Saturday or Sunday.
3. **Hotel Type:** Matters since resort hotels usually require bigger deposits making people less likely to cancel.
4. **Lead Time Buckets:** Helps group last minute bookings which almost never cancel compared to long term planners.
5. **Deposit Type:** Crucial because customers with **Non Refundable** deposits are financially locked in and rarely cancel compared to "No Deposit" bookings.

Avoiding Leakage in Feature Construction Data leakage is a huge risk when building new features. I prevented it in three ways:

1. **Dropped Future Info:** I dropped the **reservation_status** column immediately because it tells us the future outcome.
2. **Train-Only Aggregations:** When finding the **average price per country**, I only used a sample of training rows to find the mean. This stops the test set prices from leaking into the training data.
3. **Pipeline Imputing:** I used pipelines to ensure that filling missing values only happens based on the training split.

Task 8: Feature Importance and Selection

Top Features Discovered:

- **Random Forest Top 3:** risk_score, lead_time, adr
- **Chi-Square Top 3:** lead_time, adr, total_guests

Discussion: When comparing the methods I noticed a lot of overlap between **Random Forest** and **Mutual Information**. They both heavily agreed that **lead_time** and **adr** were the most important features overall. However they disagreed slightly on categorical data. I used a correlation filter to drop features that overlapped too much mathematically, like the **risk_score**, to keep the final model simple and avoid multicollinearity.

Final Comparison Table:

- FINAL COMPARISON TABLE -

	Version	Features Count	Preprocessing	Model	ROC-AUC	Notes
0	0 - Baseline	~20	Simple Imputation	Random Forest	0.76	Basic linear logic
1	1 - After Numeric Preproc	~20	RobustScaler	Random Forest	0.80	RobustScaler fixed outliers
2	2 - After Extraction/Const	~30	ColumnTransformer + Poly	Random Forest	0.86	Ratios gave big boost
3	3 - After Selection	~21	Correlation Filter (>0.85)	Random Forest	0.85	Dropped redundant/correlated ones